



Introduction to Computer Science

Unit 2: Algorithms

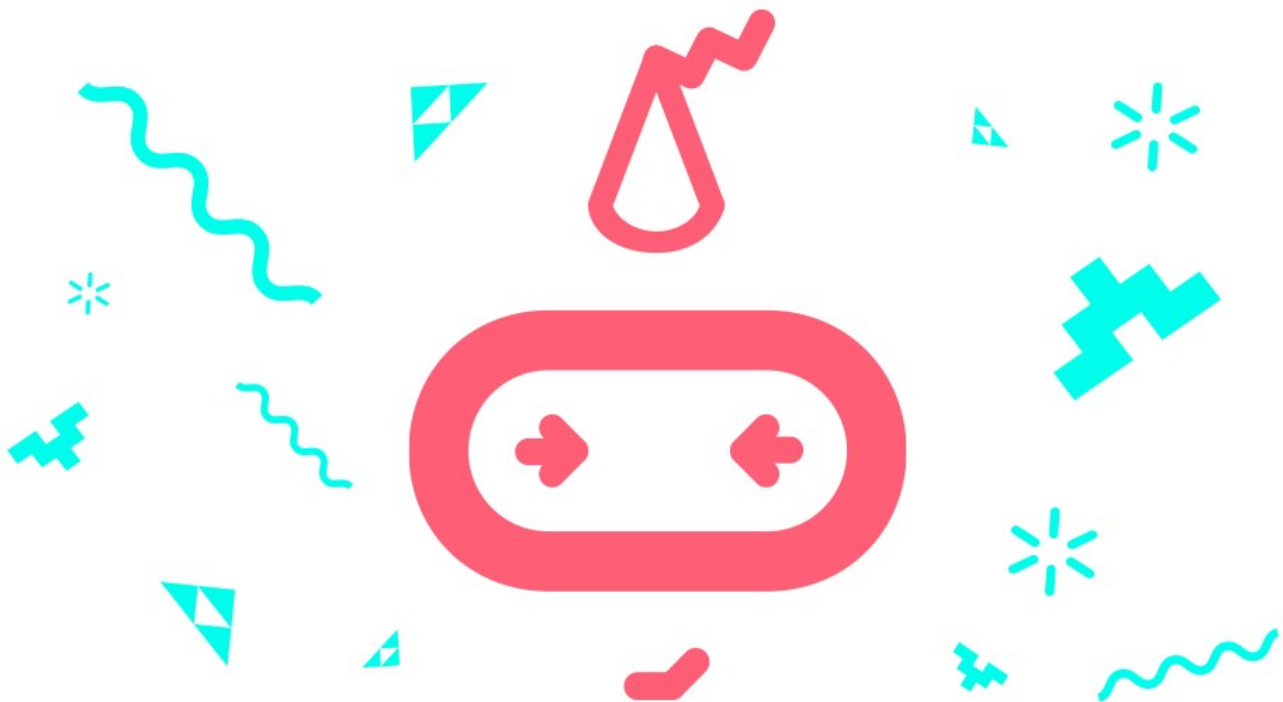
Student workbook
makecode.microbit.org



Table of Contents

Overview.....	3
---------------	---

Unit summary.....	3
Learning goals.....	3
Lesson A: How computers function.....	4
Lesson B: Code with event handlers.....	6
Lesson C: Fidget cube.....	15
Glossary of key terms.....	20



Overview

Unit summary

This unit introduces the four main components that make up a computer and the concept of input and output as it relates to programming the micro:bit. You will go through two unplugged activities relating to *algorithms* and *functions* that will create a knowledge base for your coding activity and final project. The coding activity starts with an explanation of *pseudocode* that leads to working with *events* and *event handlers* to program your micro:bit to make faces. The project incorporates all the new learning from this unit as you create your own fidget cube that responds to different inputs.

Learning goals

During this unit, you will:

- Understand the four components that make up a computer and their functions.
- Understand that the micro:bit takes input, and after processing the input, produces output.
- Learn the variety of different types of information the micro:bit takes in as input.
- Apply this knowledge by creating a micro:bit program that takes input and produces an output.

Lesson A: How computers function

Overview: Components of a computer

What is a **micro:bit**? The micro:bit was created in 2015 in the UK by the BBC and partners including Microsoft to teach computer science to students. The BBC gave away a micro:bit to every Year 7 (6th grade) student in the UK. You can think of a micro:bit as a mini computer.

What is a computer?

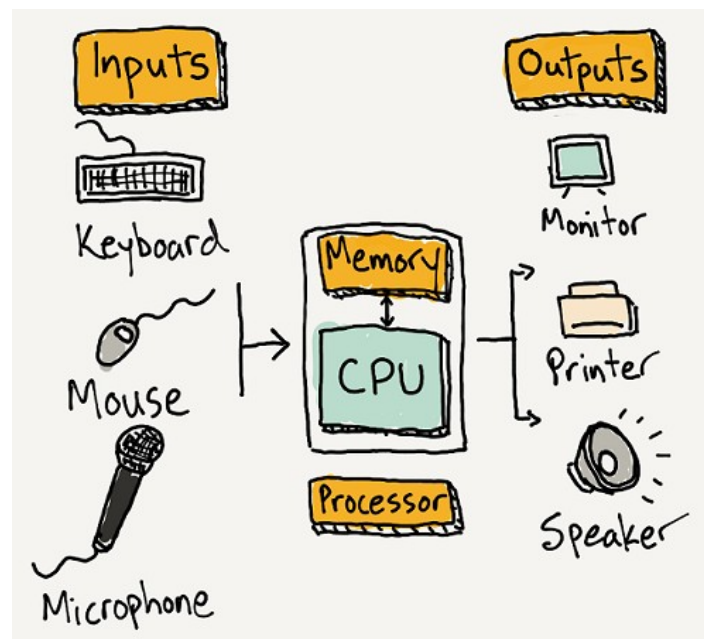
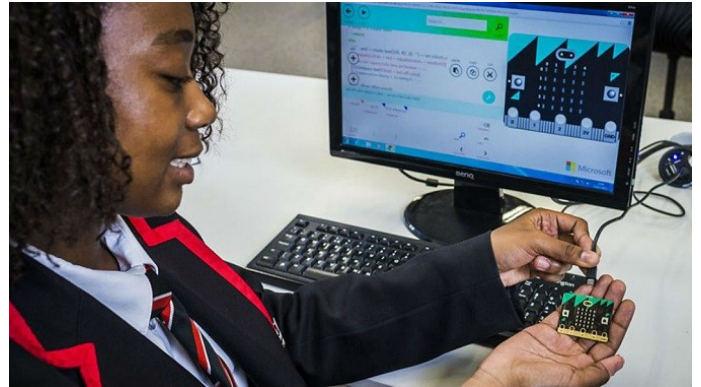
There are four main components that make up any computer:

1. **The Processor:** This is usually a small chip inside the computer, and it's how the computer processes and transforms information. Have you heard of the term "CPU"? CPU stands for Central Processing Unit. You can think of the processor as the Brains of the computer; the faster the processor, the more quickly the computer can think.
2. **The Memory:** This is how the computer remembers things. There are two types of memory:

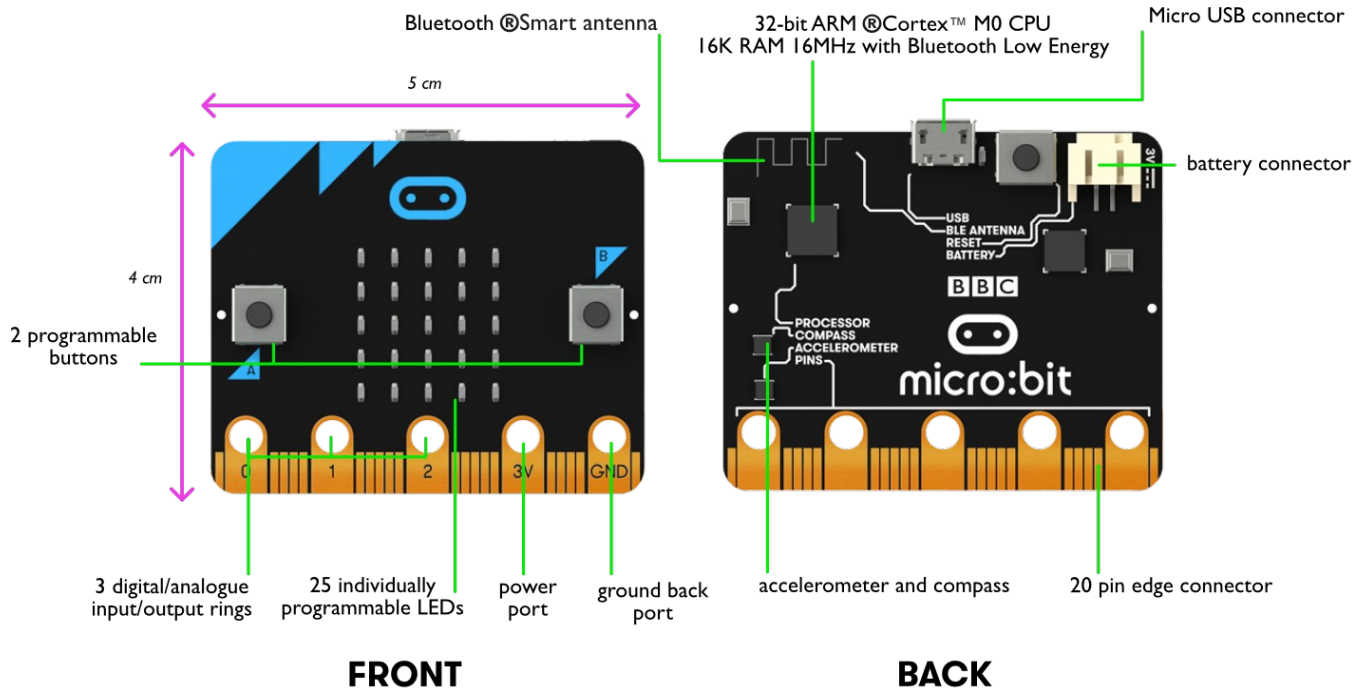
RAM (random access memory): You can think of this as the computer's short-term memory. Things that are stored here will disappear when the computer is turned off. Can you think of examples of things that are stored in our short-term memory? Things that you forget after you go to sleep?

Storage (also referred to as the computer "hard drive"): This is the computer's long-term memory, where it can store information even when power is turned off. Can you think of examples of things that are stored in our long-term memory? Things we never forget?

3. **Inputs:** This is how a computer takes in information from the world. In humans, our input comes in through our senses, such as our ears and eyes. What are some Computer Inputs? Keyboard, mouse, touchscreen, camera, microphone, game controller, scanner, etc.
4. **Outputs:** This is how a computer displays or communicates information. As humans, we communicate information by using our mouths when we talk. What are some examples of communication that don't involve talking? Blushing, sign language. What are some examples of Computer outputs? Monitor/screen, headphones/speakers, printer



Now, let's look at our micro:bit features.



All computers need electricity to power them. There are three ways to power your micro:bit:

- Connect the micro:bit to a computer through the USB port at the top
- Connect a battery pack to the battery connector
- Through the 3V Pin at the bottom (not the recommended way to power your micro:bit)

On the top left corner, you may notice that your micro:bit has a Bluetooth antenna. This means your micro:bit can communicate and send information to other micro:bits. We will learn more about this feature in Unit 10: Radio communication.

Lesson B: Code with event handlers

Coding activity: Happy face, sad face

The micro:bit itself is considered hardware. It is a physical piece of technology. In order to make use of hardware, we need to write software (otherwise known as “code” or computer programs). The software “tells” the hardware what to do—and in what order to do it using algorithms. *Algorithms* are sets of computer instructions.

In this activity, you will discover how to use the micro:bit buttons as input devices and write code that will make something happen on the screen as output. You will also learn about pseudocode, the MakeCode tool, event handlers, and commenting code.

Pseudocode

What do you want your program to do? The first step in writing a computer program is to create a plan for what you want your program to do. Write out a detailed step-by-step plan for your program. Your plan should include what type of information your program will receive, how this input will be processed, what output your program will create, and how the output will be recorded or presented. Your writing does not need to be written in complete sentences nor include actual code. This kind of detailed writing is known as pseudocode.

Pseudocode is like a detailed outline or rough draft of your program. Pseudocode is a mix of natural language and code.

For the program you will write, the pseudocode might look like this:

- Start with a blank screen
- Whenever the user presses button A, display a happy face
- Whenever the user presses button B, display a sad face

Use this space to write your pseudocode

Microsoft MakeCode

Now that you have a plan for your program in the form of pseudocode, let's start creating the real program in Microsoft MakeCode. Remember, the MakeCode tool is called an IDE (Integrated Development Environment) and is a software application that contains everything a programmer needs to create, compile, run, test, and even debug a program.

1. In Microsoft MakeCode, start a new project.

Event handlers

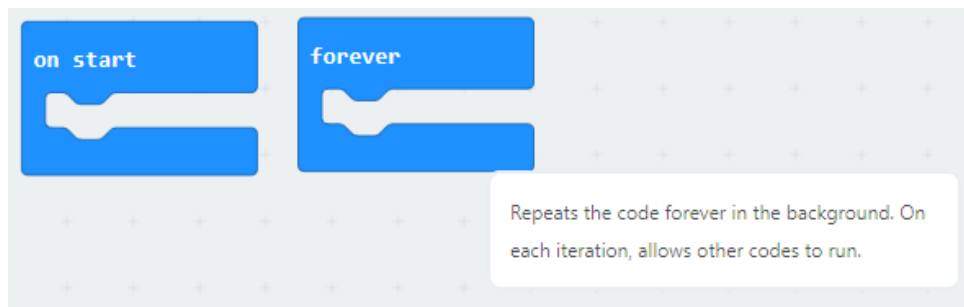
When you start a new project, there will be two blue blocks, 'on start' and 'forever' already in the coding Workspace. These two blocks are *event handlers*.

In programming, an *event* is an action done by the user, such as pressing a key or clicking a mouse button. An *event handler* is a routine that responds to an event. A programmer can write code telling the computer what to do when an event occurs.

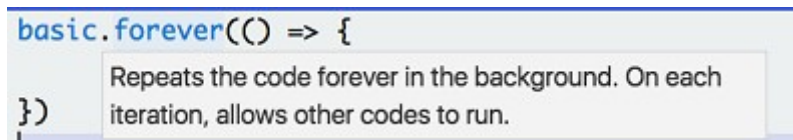
Tool tips

Experiment with the following options.

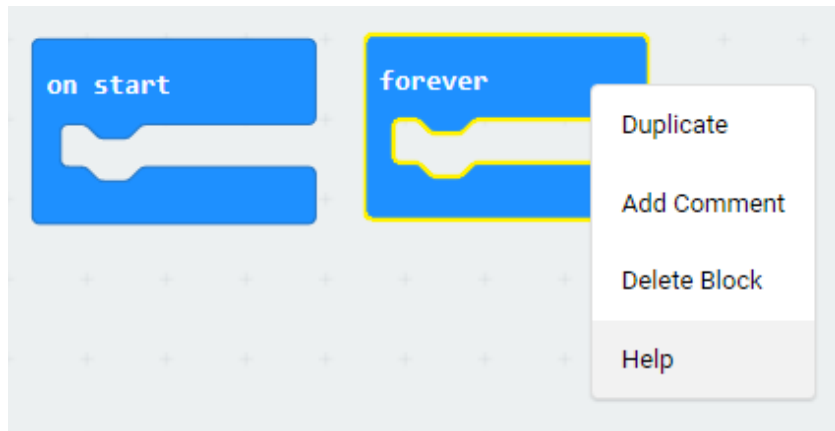
2. Block descriptions: Hover over any block until a hand icon appears and a small text box will pop up telling you what that block does. You can try this now with the 'on start' and 'forever' blocks.



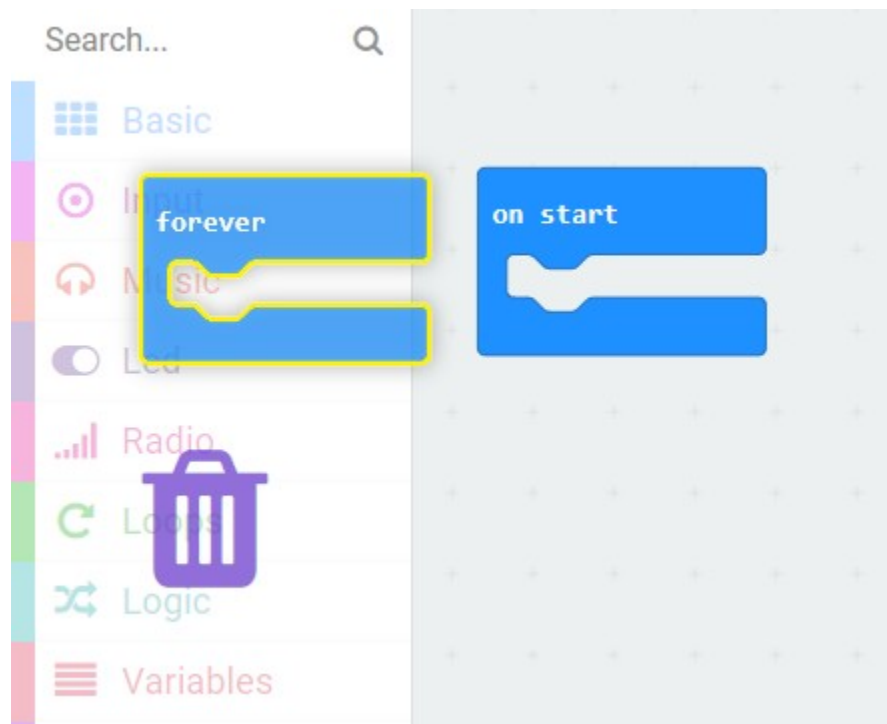
Hovering over the code in JavaScript has the same effect.



Help: You can also right-click on any block and select Help to open the reference documentation.

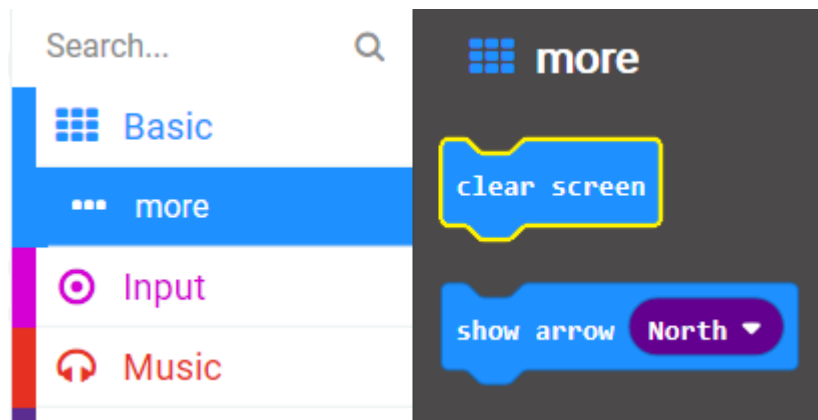


3. Deleting blocks: Select the 'forever' block and drag it left to the Toolbox area. You should see a garbage can icon appear. Let go of the block and it should disappear. You can drag any block back to the Toolbox area to delete it from the coding Workspace. You can also remove a block from the coding Workspace by:
- Hovering over the block, right-clicking, and selecting Delete Block, or
 - Selecting the block and then pressing the "delete" key on your keyboard (or command-X on a mac).

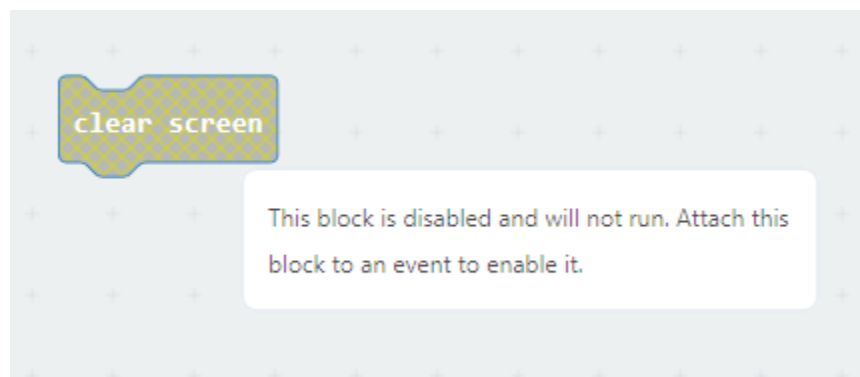


Clear screen

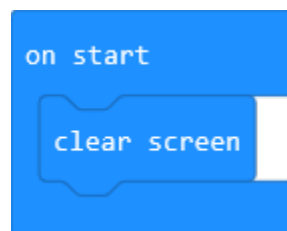
- Looking at our pseudocode, we want to make sure to start a program with a clear screen. We can do this by going to the Basic menu, selecting ... more, and choosing a 'clear screen' block.



- Drag the 'clear screen' block to the coding Workspace. Notice that the block is grayed out. If you hover over the grayed out block, a pop-up text box will appear letting you know that since this block is not attached to an event handler block, it will not run.



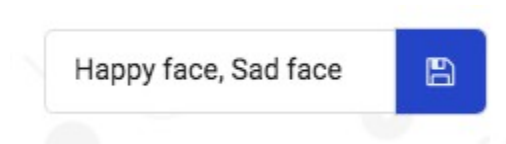
- Go ahead and drag the 'clear screen' block into the 'on start' block. Now the block is no longer grayed out, indicating that it will run when the event occurs, i.e., the program starts.



Save early, save often!

You now have a working program running on the micro:bit simulator! As you write your program, MakeCode will automatically compile and run your code on the simulator. The program doesn't do much at this point, but before we make it more interesting, we should name our program and save it.

7. On the bottom left of the application window, to the right of the Download button is a text box in which you can name your program. After naming your program, select the save button to save it.



Important: Whenever you write a significant piece of code or just every few minutes, you should save your code. Giving your code a meaningful name will help you find it faster and let others know what your program does.

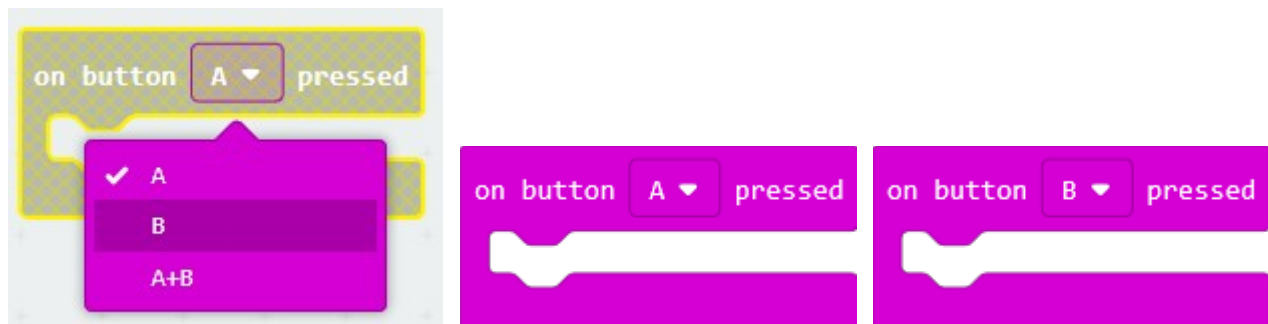
More event handlers

Now to make your program a bit more interesting by adding two more event handlers.

8. From the Input menu, drag two 'on button A pressed' blocks to the coding Workspace. Notice that the second block is grayed out. This is because, right now, they are the same block, both "listening" for the same event 'on button A pressed'.



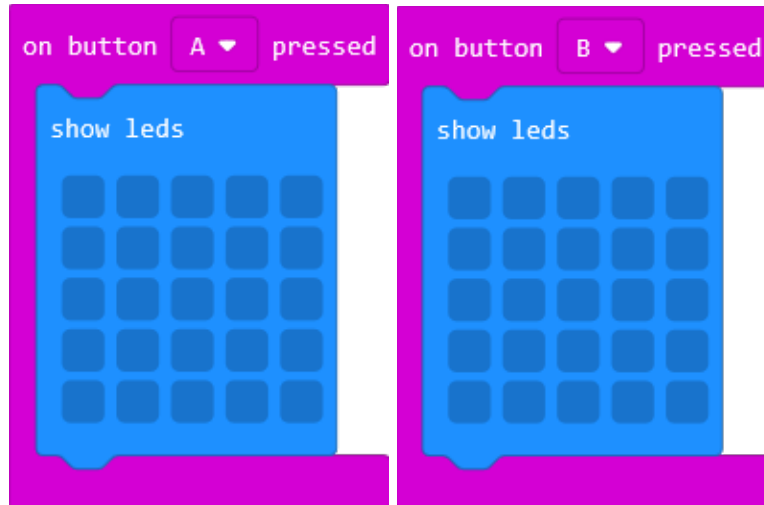
9. Leave the first block alone for now, and using the drop-down menu within the second block, change the A to B. Now this block will no longer be grayed out, as it is now listening for a different event, 'on button B pressed'.



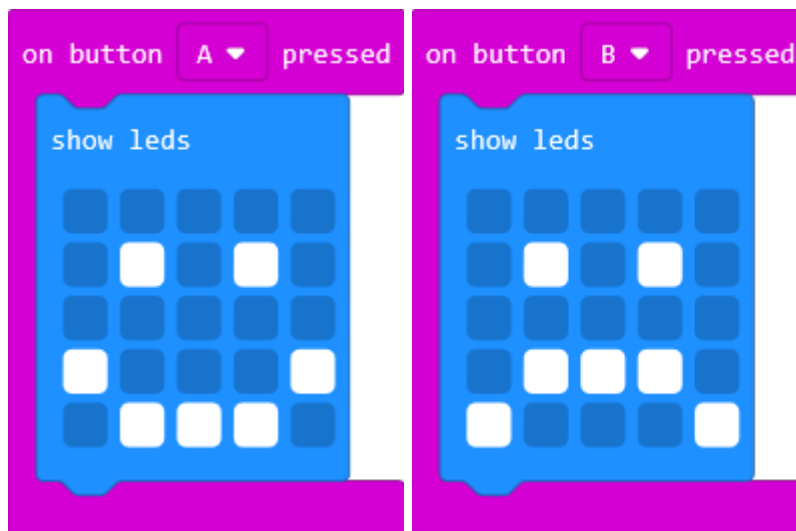
Show LEDs

Now we can use our LED lights to display different images depending on what button the user presses.

10. From the Basic Toolbox, drag two 'show leds' blocks to the coding Workspace. Place one 'show leds' block into the 'on button A pressed' event handler and the second 'show leds' block into the 'on button B pressed' event handler.



11. Select the individual little boxes in the 'show leds' block that is in the 'on button A pressed' event handler to create the image of a happy face. Select the individual little boxes in the 'show leds' block that is in the 'on button B pressed' event handler to create the image of a sad face.

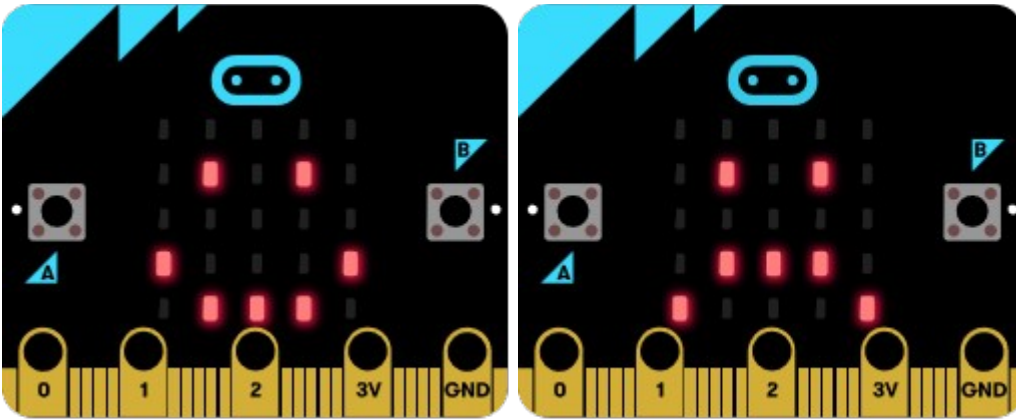


Test your program!

12. In the Simulator, press button A and then button B to see the output produced by your code.

Feel free to play around with turning LEDs on or off in the 'show leds' blocks until you get the images you want.

Remember to save your code.

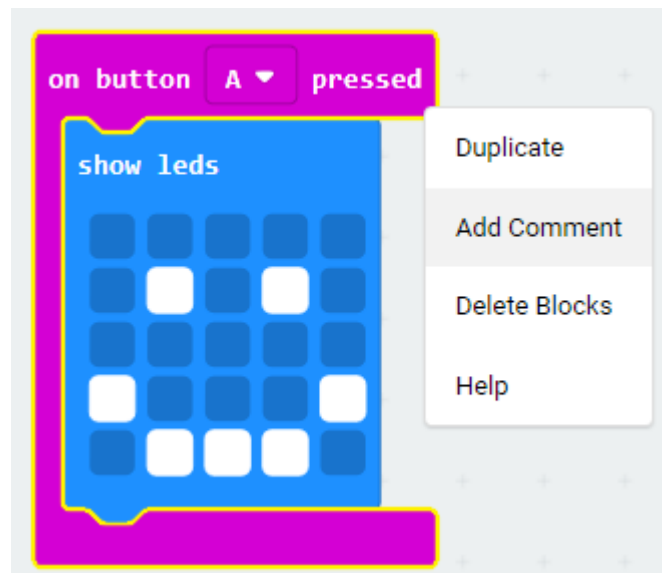


Commenting on your code

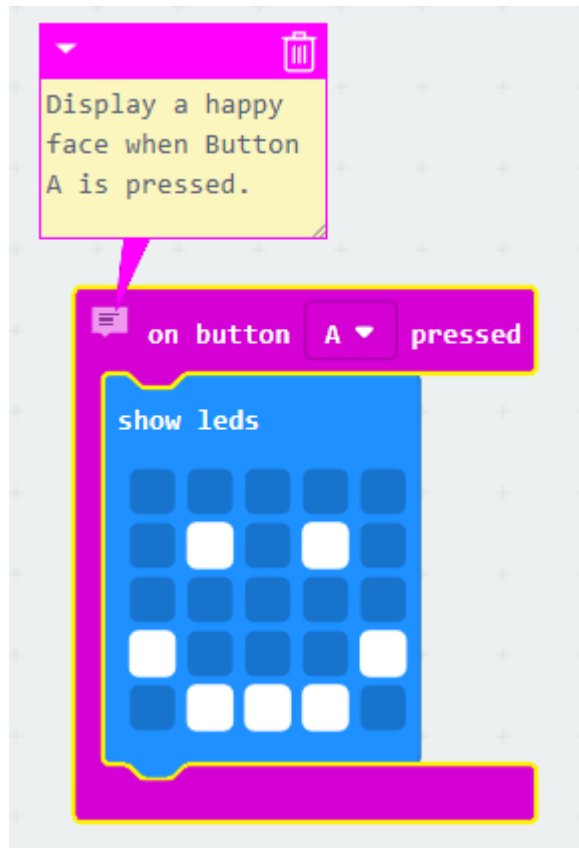
It is good practice to add comments to your code. Comments can be useful in a number of ways. Comments can help you remember what a certain block of code does and/or why you chose to program something the way you did. Comments also help others reading your code to understand these same things.

To comment on a block of code:

- Right-click on the icon that appears before the words on a block.
- A menu will pop up. Select 'Add Comment'.



- This will cause a question mark icon to appear to the left of the previous icon.
- Select the question mark and a small yellow box will appear into which you can write your comment.



- Select the question mark icon again to close the comment box when you are done.
- Select the question mark icon whenever you want to see your comment again or to edit it.

In JavaScript, you can add a comment by using two forward slashes, then typing your comment. The two forward slashes tell JavaScript that the following text (on that same line) is a comment.

```
// Display a happy face when button A is pressed.
```

Cleaning up!

13. Clean up your coding Workspace before you do a final save. What does this mean?

- It means that only the code and blocks that you are using in your program are still in the workspace.
- Remove (delete) any other blocks that you may have dragged into the coding workspace as you were experimenting and building your program.

Save and download

14. Now that your code is running just fine in the Simulator, is commented, and your coding Workspace is "clean," save your program, download it to your micro:bit, play and enjoy!

Lesson C: Fidget cube

Activity: Fidget cube

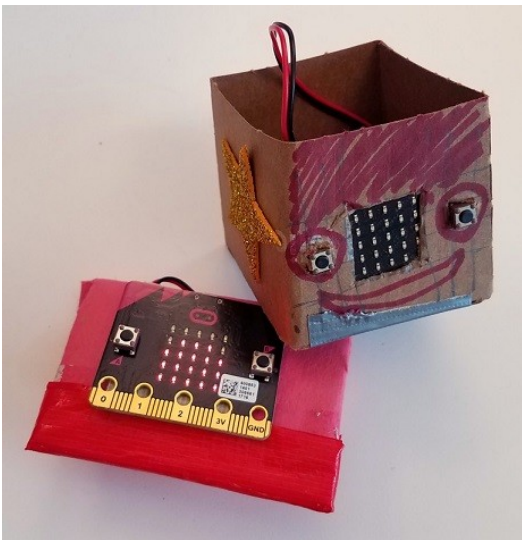
A fidget cube is a little cube with something different that you can manipulate on each surface. There are buttons, switches, and dials, and people who like to “fidget” find it relaxing to push, pull, press, and play with it. In this project, students are challenged to turn the micro:bit into their very own “fidget cube”.

This project is to make a fidget cube out of the micro:bit and create a unique output for each of the following inputs:

- on button A pressed
- on button B pressed
- on button A+B pressed
- on shake

See if you can combine a maker element similar to what you created in Unit 1 by providing a holder for the micro:bit that holds it securely when you press one of the buttons.

Fidget cube example



Project mods

- Add more inputs and outputs; use more than four different types of input.
- Try to use other types of output (other than LEDs) such as sound!

Challenge

For an extra challenge, you can try coding this in JavaScript or Python. You can select the Blocks tab at the top of the MakeCode editor window to switch back and forth between Blocks, JavaScript, and Python

Project expectations

Follow the design thinking approach and make sure your project meets the required specifications:

- Uses at least four different inputs
- Uses at least four different outputs
- Uses event handlers in a way that is integral to the program
- The program compiles and runs as intended and uses meaningful comments
- Includes the written Reflection Diary entry (which we'll talk about after you complete your project)

The design thinking process:

1. **Empathize** by learning more about your target audience.
2. **Define**: Understand and identify your audience's problems or needs.
3. **Ideate**: Brainstorm several possible creative solutions.
4. **Prototype**: Construct rough drafts or sketches of your ideas.
5. **Test** your prototype solutions, and refine until you come up with the final version.

Use this space to track your design thinking process:

Project scoring rubric

Assessment elements	1	2	3	4
---------------------	---	---	---	---

Inputs	Fewer than two different inputs are successfully implemented.	At least two different inputs are successfully implemented.	At least three different inputs are successfully implemented.	At least four different inputs are successfully implemented.
Outputs	Fewer than two different outputs are successfully implemented.	At least two different outputs are successfully implemented.	At least three different outputs are successfully implemented.	At least four different outputs are successfully implemented.
micro:bit program	micro:bit program lacks all of the required elements.	micro:bit program lacks two of the required elements.	micro:bit program lacks one of the required elements.	micro:bit program: ■ Uses event handlers in a way that is integral to the program ■ Compiles and runs as intended ■ Uses meaningful comments in code

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- What problem did you solve or why did you decide to create this project?
- What kind of input and output did you decide to use?
- What does your program do? Describe how your program works (what the cause and effect are).
- Include at least one screenshot of your program working.
- Publish your MakeCode program and include the URL.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements.

Glossary of key terms

Algorithm: A set of (often repeated) steps used to solve a problem. The set of steps for doing long division of a number is an algorithm.

Event: Something that happens outside a program (like a screen tap or mouse click) that the program can respond to.

Event handler: Part of a program that runs when a specific event happens (it “handles” the event). In MakeCode, an event handler block looks like a square with a gap in the middle and usually starts with the word “on.”

Function: A self-contained set of instructions for performing a specific task within a computer program. Most objects have multiple functions associated with them.

Input: This is how a computer takes in information from the world. For example: keyboard, mouse, touchscreen, camera, microphone, game controller, scanner.

Memory: How the computer remembers things. There are two types of memory:

- **RAM** (random access memory): The computer’s short-term memory. Things that are stored here will disappear when the computer is turned off.
- **Storage** (also referred to as the computer “hard drive”): This is the computer’s long-term memory, where it can store information even when power is turned off.

Output: This is how a computer displays or communicates information. For example: monitor/screen, headphones/speakers, printer.

Processor: This is a chip inside the computer, and it’s how the computer processes and transforms information.

Pseudocode: A detailed outline or rough draft of a program. Pseudocode is a mix of natural language and code.